Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- İlişkisel Model Kısıtları (Integrity Constraints)
  - Anahtar Kısıtı (Key Constraint)
  - Varlık Bütünlük Kısıtı (Entity Integrity Constraint)
  - İma Bütünlük Kısıtı (Referential Integrity Constraint)
  - Domain Kısıtı
- Veri Tanımlama Dili (DDL) ile Tablo Oluşturma
  - CREATE TABLE komutları
  - PRIMARY KEY, FOREIGN KEY, UNIQUE tanımlamaları
  - ALTER TABLE ile kısıt ekleme
  - DROP TABLE ve sıralama kuralları
- Bütünlük İhlalleri ve Çözüm Yolları
  - INSERT, DELETE, UPDATE işlemlerinde ihlaller
  - Cascade, Restrict, Reject, Set Null, Set Default seçenekleri
  - onDelete ve onUpdate davranışları
- İlişkisel Modele Dönüşüm Kuralları (ER → İlişkisel Model)
  - Güçlü varlık setlerinin tabloya dönüştürülmesi
  - Zayıf varlık setlerinin tabloya dönüştürülmesi
  - Birebirlik (1:1) bağıntıların dönüştürülmesi
  - Birenlilik (1:N) bağıntıların dönüştürülmesi
  - Ene-enlik (M:N) bağıntıların dönüştürülmesi
  - Çok değerli (Multi-Valued) niteliklerin dönüştürülmesi
  - 3 dereceli bağıntıların dönüştürülmesi
  - Rekürsif bağıntıların dönüştürülmesi
- Genişletilmiş İlişkisel Model (EIR) Dönüşümleri
  - Specialization / Generalization dönüşümleri
  - Disjoint ve Overlap durumları
  - Total ve Partial participation durumları
  - Farklı dönüşüm stratejileri (8A, 8B, 8C, 8D)
- İlişkisel Cebire Giriş
  - Tek tablo operatörleri: Select (σ), Project (π), Sort, Rename, Extend, Aggregate
  - İki tablo operatörleri: Union, Intersection, Set Difference, Division, Cartesian Product, Join
  - İlişkisel cebir ağacı ve çalıştırma sırası
  - Kümeneme fonksiyonları (SUM, MIN, MAX, COUNT, AVG)

Hocanın Özellikle Vurguladığı Kısımlar

- Bütünlük kısıtlarının isimlerinin (Anahtar Kısıtı, Varlık Bütünlük, İma Bütünlük) mutlaka öğrenilmesi gerektiği
  - Sınavlarda bu isimler ve ne manaya geldikleri soruluyor
- İlişkisel modele dönüşüm kurallarının ezber değil, sebepleriyle anlaşılması gerektiği
  - Geçmiş senelerde %70-80 başarı oranı ancak bu şekilde yakalanabiliyor
- Birebirlik bağıntıda yabancı anahtarın total participation olan tarafa eklenmesi gerektiği
  - Aksi halde çok sayıda NULL değer ortaya çıkar, performans düşer
- Birenlilik bağıntıda yabancı anahtarın mutlaka “en” (N) olan tarafa eklenmesi gerektiği

  - Bir olan tarafa eklenirse ilişki takip edilemez
- Multi-valued nitelikler için mutlaka ayrı tablo oluşturulması gerektiği
  - Aynı tabloda virgülle ayırarak yazmak ilişkisel modelin formal tanımına aykırı (atomik olma şartı)
  - Veri tekrarı ve update anomalilerine yol açar
- Şemanın iyi bilinmesi gerektiği
  - Hangi tablo nereye referans ediyor bilmek, sorgu yazmak ve integrity analizini yapmak için şart
- DDL komutlarının sıralamasının önemli olması
  - Referans edilen tablo önce oluşturulmalı, sonra ALTER TABLE ile yabancı anahtar eklenmeli
  - Aynı durum DROP TABLE için de geçerli
- İlişkisel cebirin formal bir dil olmadığı, ancak SQL'in arka planda ilişkisel cebire dönüştürülüp optimize edildiği
  - İlişkisel cebir ağacı: yapraklarda tablolar, dallarda operatörler, kökte sonuç kümesi

Kısa Tekrar Notları

- İlişkisel modelin 3 temel kısıtı: Anahtar, Varlık Bütünlük, İma Bütünlük
- Superkey: Tablodaki satırları ayırt edebilen nitelik/nitelik grubu
- Key (Anahtar): Minimal superkey
- Candidate Key: Birden fazla olabilir, bunlardan biri Primary Key olarak seçilir
- Primary Key: NULL olamaz, unique olmalı, mümkünse minimal nitelik sayısı tercih edilir
- Secondary Key: Primary Key dışındaki nitelikler üzerinde arama yapılır
- Varlık Bütünlük: Primary Key'in herhangi bir kısmı NULL olamaz
- İma Bütünlük: Yabancı anahtar ya NULL ya da işaret ettiği tabloda mevcut bir kayıt olmalı
- Foreign Key tanımlamasında: ON DELETE ve ON UPDATE davranışları belirlenir
- CASCADE: Silme/güncelleme zincirleme olarak diğer tabloya da yansır
- RESTRICT: İşlemi engeller
- REJECT: Silmeyi/güncellemeyi geri çevirir
- SET NULL: Yabancı anahtarı NULL yapar
- SET DEFAULT: Yabancı anahtarı varsayılan değere çeker
- INSERT işleminde: Domain, Key, Entity Integrity, Referential Integrity hepsi kontrol edilir
- DELETE işleminde: Sadece Referential Integrity ihlali olabilir
- UPDATE işleminde: Güncellenen niteliğin türüne göre ihlaller değişir
  - Primary Key güncellemesi: Key + Referential Integrity ihlali
  - Foreign Key güncellemesi: Referential Integrity ihlali
  - Sıradan nitelik güncellemesi: Sadece Domain ihlali
- Güçlü varlık → Tablo (nitelikler taşınır, PK belirlenir)
- Zayıf varlık → Tablo (owner'ın PK'si FK olarak gelir, PK = owner PK + partial key)
- 1:1 bağıntı → 3 farklı yaklaşım: FK ile, merge ile, veya ayrı tablo ile
- 1:N bağıntı → N olan tarafa FK eklenir
- M:N bağıntı → Yeni bir tablo oluşturulur, her iki tablonun PK'si FK olarak gelir
- Multi-valued nitelik → Ayrı tablo (owner FK + nitelik), PK = owner FK + nitelik
- 3 dereceli bağıntı → Yeni tablo, N olan tarafların PK'leri birleşerek tablonun PK'sini oluşturur
- Disjoint + Total → Sadece subtype'lar için tablo (super type tablosu gerekmez) → 8B
- Disjoint + Partial/Total → Her subtype için ayrı tablo + super type tablosu → 8A
- Disjoint → Tek tablo + discriminator (type) niteliği → 8C
- Overlap → Tek tablo + birden fazla flag (Boolean) niteliği → 8D
- İlişkisel cebir: σ (selection/yatay), π (project/dikey), ∪, ∩, −, ÷, ×, ⋈ (join)
- Selection: Satır seçimi (SQL'deki SELECT değil, WHERE kısmına karşılık gelir)
- Projection: Sütun seçimi, tekrar eden satırlar elimine edilir (set özelliği)
- Cartesian Product: İki tablonun her satırını diğerinin her satırıyla eşleştirir
- Join: Cartesian Product + filtre (yükleme göre anlamlı satırlar seçilir)
- Çoğu zaman JOIN, yabancı anahtar-primary key eşitliği üzerine kurulur

Detaylı Açıklamalar

- **Tablo Oluşturma Sıralaması:** İki tablo birbirine referans ediyorsa (örneğin Employee ve Department), her iki tabloyu da aynı anda oluşturmak mümkün değildir. Önce temel tablo (referans edilmeyen veya daha az referans içeren) oluşturulur, ardından ALTER TABLE komutuyla yabancı anahtarlar eklenir. Büyük veritabanlarında bu sıralamayı otomatik olarak yapan araçlar mevcuttur (PostgreSQL'de mevcut).
- **Veri Girişi Sıralaması:** Tablolar oluşturduktan sonra veri girerken de referans bütünlüğüne dikkat edilmelidir. Örneğin Employee tablosuna ilk kayıt girilirken Department tablosunda henüz kayıt olmadığı için DNO ve SuperSSN alanlarına NULL girilir. Daha sonra Department kayıtları girildikten sonra UPDATE komutlarıyla bu NULL değerler gerçek değerlerle değiştirilir.
- **Employee Tablosundan Satır Silme Analizi:** Employee tablosundan bir satır silindiğinde kontrol edilmesi gerekenler: (1) Dependent tablosu bu employee'e referans ediyor mu? Ediyorsa CASCADE ile bağımlı kayıtlar da silinmeli (gerçek hayata uygun: işçi işten çıkınca aile fertleri de sistemden çıkar). (2) Department tablosunda ManagerSSN olarak bu employee'e referans var mı? (3) Works_On tablosunda bu employee'in proje kayıtları var mı? (4) Employee tablosundaki SuperSSN (self-referencing) - bu employee'i supervisor olarak işaret eden başka kayıt var mı? Her bir referans için yabancı anahtar tanımındaki ON DELETE davranışı (CASCADE, SET DEFAULT, SET NULL, RESTRICT) belirleyici olur.
- **Birebirlik Bağıntıda Merge Yaklaşımı:** Birebirlik bağıntıda iki tablo birleştirilip tek tablo yapılabilir (merge). Ancak bu yaklaşım ancak her iki taraf da total participation ise ve nitelik sayısı az ise uygundur. Aksi halde çok sayıda NULL değer ortaya çıkar. En iyi yaklaşım çoğu zaman yabancı anahtar ile bağlamaktır.
- **Birebirlik Bağıntıda Yabancı Anahtar Yönü:** Birebirlik bağıntıda yabancı anahtar her iki tarafa da eklenebilir ancak total participation olan tarafa eklemek tercih edilir. Böylece NULL değer görülmez. Örneğin Department (total) - Employee (partial) arasında "manages" bağıntısı varsa, ManagerSSN yabancı anahtarı Department tablosuna eklenir. Böylece her departmanın mutlaka bir yöneticisi olduğu için NULL olmaz.
- **Multi-Valued Nitelik Problemi:** Department'ın birden çok lokasyonu olabilir. Eğer lokasyonları Department tablosuna "İstanbul,Ankara,İzmir" şeklinde yazarsak, ilişkisel modelin atomik veri şartını ihlal ederiz. Ayrıca manager gibi nitelikler her lokasyon için tekrar eder, bu da veri tekrarı ve update anomalisi yaratır. Doğru çözüm: Dept_Locations adında ayrı bir tablo oluşturmak (DNumber FK + DLocation), PK = (DNumber, DLocation).
- **EIR Dönüşüm Stratejileri:** Specialization/Generalization yapılarını ilişkisel modele dönüştürürken 4 farklı yaklaşım vardır:
  - **8A (Her sınıf için ayrı tablo):** En kolay, ezbere çözüm. Super type ve her subtype için ayrı tablo. Subtype tablolarında super type'ın PK'si yabancı anahtar olur.
  - **8B (Sadece subtype'lar için tablo):** Eğer participation total ise super type tablosuna gerek yoktur. Tüm varlıklar zaten subtype'larda bulunur. Super type'ın nitelikleri her subtype tablosuna taşınır.
  - **8C (Tek tablo + discriminator - Disjoint için):** Tüm nitelikler tek tabloda toplanır. Hangi subtype'a ait olduğunu belirlemek için bir discriminator (type) niteliği eklenir. Disjoint ise bir kayıt sadece bir tipe ait olabilir.
  - **8D (Tek tablo + flag'ler - Overlap için):** Her subtype için bir Boolean flag niteliği eklenir. Overlap durumunda bir kayıt birden fazla tipe ait olabilir, bu yüzden birden fazla flag True olabilir. Dezavantaj: Çok fazla NULL değer ortaya çıkabilir.
- **İlişkisel Cebir Ağacı:** SQL sorgusu arka planda ilişkisel cebire dönüştürülür ve bir ağaç yapısında temsil edilir. Ağacın yapraklarında tablolar bulunur, dallarda ilişkisel operatörler (σ, π, ⋈ vb.) yer alır. Kök düğüm sorgunun sonucunu verir. Bu ağaç üzerinde optimizasyon yapılır. Execution order ağaçtan okunur.
- **Join Mantığı:** Cartesian Product iki tablonun tüm satır kombinasyonlarını üretir. Bu ürünün çoğu anlamsızdır. Join, bu anlamsız satırları bir yükleme (predicate) göre filtreler. Çoğu zaman bu yükleme yabancı anahtar = primary key eşitliğidir. Örneğin Employee

× Project çarpımında sadece Employee.Project = Project.Code şartını sağlayan satırlar anlamlıdır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.